정규교과 3/4

머신러닝 모델링 및 성능평가

Day 7 · 트리 · 앙상블 → 특성 중요도 → 과적합 · 교차검증

2026.10.27 (화) | Decision Tree · Random Forest · XGBoost

한로 강민준

REVIEW

어제 우리가 배운 것

모델: 선형회귀·로지스틱회귀·KNN의 예측 방식을 비교했다.

전처리: 같은 분할 ID를 유지하고 train에서만 규칙을 학습했다.

평가: 회귀 RMSE와 분류 F1을 구분하고 선택 근거·한계를 기록했다.

오늘은 같은 문제에 트리·앙상블을 연결하고, train 내부 교차검증을 추가한다.

공식 참고: scikit-learn · Pipeline / Cross-validation

한로 강민준

3분 확인 퀴즈 · Day6 복습

Q1. 모델을 바꾸면 train·valid·test도 다시 나눌까?

정답·해설 아니다. 같은 문제에서는 분할 ID와 순서를 유지한다.

Q2. KNN의 스케일러는 어떤 자료로 학습하는가?

정답·해설 train만 사용한다. valid에서는 transform만 수행한다.

Q3. valid 최고 점수는 최종 일반화 성능인가?

정답·해설 후보 선택에 쓴 점수다. 보관한 test의 최종 평가와 구분한다.

공식 참고: scikit-learn · Common pitfalls

한로 강민준

오늘의 목표와 8교시 운영

교시	핵심 내용	설명·시연	실습·리뷰
1	Decision Tree 원리 · 분류·회귀	30분	20분
2	깊이 · 잎 크기 · 과적합 진단	25분	25분
3	Random Forest · 다양한 트리의 결합	25분	25분
4	특성 중요도 · 해석의 한계	25분	25분
5	XGBoost · 순차 보정 · 구현	30분	20분
6	과적합 제어 · train 내부 5-fold CV	25분	25분
7	모델 비교 · 프로젝트 통합	25분	25분
8	검증 · 실험 보고서 · 회고	15분	35분

50분 × 8교시 = 400분 · 설명/시연 200분 + 실습/리뷰 200분 · 교시 사이 10분 휴식

한로 강민준

오늘의 범위와 다음 수업의 경계

오늘 · 10/27

질문으로 나누는 트리
여러 트리를 묶는 앙상블
중요도의 의미와 한계
과적합 진단·5-fold 적용
앙상블 실험 보고서

다음 수업 · 10/28

지표·임계값·교차검증 심화
Feature Engineering·K-Means
최종 Pipeline과 실습평가
선택 정책을 고정한 뒤 test 평가

오늘의 최고 점수는 실험 결과이지, 서비스 배포를 승인하는 성능 인증이 아니다.

공식 참고: scikit-learn · Cross-validation

한로 강민준

기존 가상환경 확인 · 오늘은 CPU로 실습

```
python --version
python -m pip show scikit-learn xgboost
python -m pip check

# xgboost가 없을 때: 강의 검증 버전 예시
python -m pip install "xgboost==3.1.3"

python -c "import xgboost; print(xgboost.__version__)"
```

점검 포인트

VS Code 실행기
프로젝트 .venv 선택

pip check
의존성 충돌 확인

CPU · hist · n_jobs=1

실행 검증 환경

```
Python 3.13.5
sklearn 1.8.0
XGBoost 3.1.3
GPU 불필요
```

공식 참고: XGBoost · Installation

한로 강민준

공통 데이터와 예측 시점은 그대로 유지

구분	사용 내용	역할·주의점
입력 X · 4열	input_chars · allowed_tools task_type · priority	실행 전에 확인할 수 있는 값
분류 정답	failed · 0=성공, 1=실패	실패 클래스 F1을 주 지표로 사용
회귀 정답	elapsed_ms	RMSE의 단위는 ms
금지 입력	failed · elapsed_ms retry_count · status · request_id	정답·실행 후 정보·식별자는 X에서 제외
고정 분할	train 60 · valid 20 · test 20	원본·분할 ID·분할 내 행 순서 재사용

수업용 합성 요청 100건이다. 실제 서비스의 성능·고객 특성을 대표하지 않는다.

공식 참고: scikit-learn · Common pitfalls

오늘의 평가 규칙 · CV와 valid를 구분

train 60

5-fold로 내부 비교
각 fold에서 전처리 학습

→

valid 20

이미 사용한 진단 자료
오류·기존 결과 비교

→

test 20

오늘도 보관
최종 선택 후 확인

후보·지표는 먼저 정한다. CV와 valid의 반복 조회가 선택 편향을 없애 주지는 않는다.

공식 참고: scikit-learn · Cross-validation / Common pitfalls

한로 강민준

Decision Tree · 질문을 나누어 예측한다

한 번에 한 특성의 조건을 검사하고, 조건에 따라 자료를 두 부분으로 나눈다.

루트는 첫 질문, 내부 노드는 다음 질문, 잎은 최종 예측 위치다.

분류는 잎의 클래스 분포를, 회귀는 잎의 대표 숫자를 사용한다.

질문은 사람이 모두 적는 것이 아니라, 학습 자료의 기준으로 선택한다.

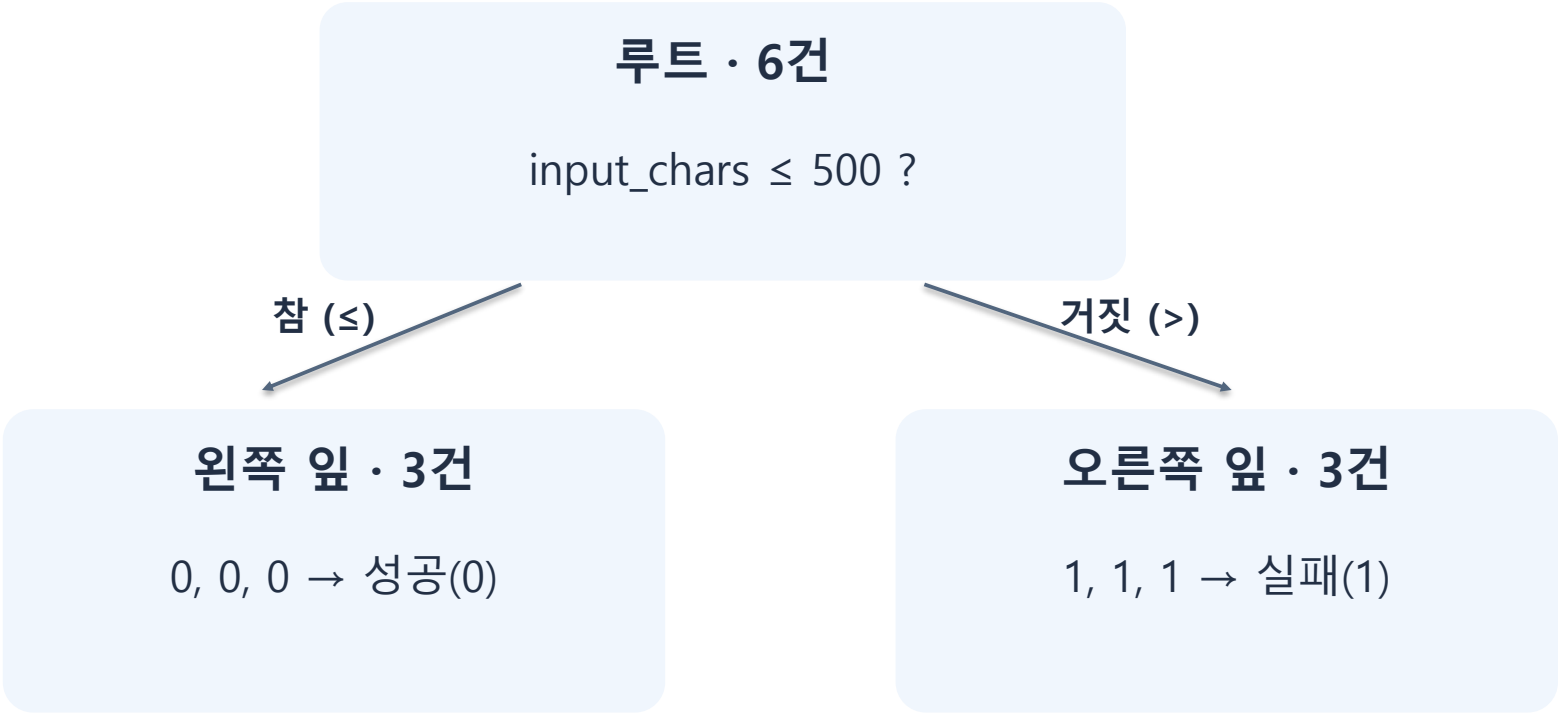
실무 예: 실행 전 요청의 실패 위험을 설명 가능한 조건으로 점검한다.

공식 참고: scikit-learn · Decision Trees

한로 강민준

작은 예제 · “글자 수 ≤ 500”이라는 질문

input_chars / failed	
100	0
200	0
300	0
700	1
800	1
900	1



fit: 좋은 질문을 찾는다. predict: 이미 찾은 질문을 따라 잎으로 이동한다.

공식 참고: scikit-learn · DecisionTreeClassifier

한로 강민준

질문 선택 · 섞인 정도가 얼마나 줄었는가?

항목	계산	해석
부모 노드	Gini = 1 − 0.5^2 − 0.5^2 = 0.5	성공 3건·실패 3건이 섞여 있음
후보 A · 250	왼쪽 2건·오른쪽 4건 분할 후 가중 Gini = 0.25	감소량 = 0.5 − 0.25 = 0.25
후보 B · 500	왼쪽 3건·오른쪽 3건 분할 후 가중 Gini = 0	감소량 = 0.5 − 0 = 0.5
이번 선택	두 후보 중 500의 감소량이 더 큼	전체 최적을 한 번에 찾는다는 뜻은 아님

Gini는 분류 노드의 혼합 정도다. 시험 점수나 회귀 오차와 같은 지표가 아니다.

공식 참고: scikit-learn · Decision Trees / DecisionTreeClassifier

한로 강민준

첫 분류 트리 · 조건과 예측 확인

```
X=np.array([[100],[200],[300],[700],[800],[900]])
y=np.array([0,0,0,1,1,1])
tree=DecisionTreeClassifier(max_depth=1,random_state=42)
tree.fit(X,y)
print('임계값:',tree.tree_.threshold[0])
print('예측:',tree.predict([[250],[850]]).tolist())
print('확률:',tree.predict_proba([[250],[850]]).tolist())
print('깊이 / 잎:',tree.get_depth(),tree.get_n_leaves())
```

E01 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

max_depth=1
질문 깊이를 1로 제한

fit / predict
학습 / 새 입력의 예측

get_n_leaves()
최종 잎의 개수

실행 결과 · 요약

```
임계값: 500.0
예측: [0, 1]
깊이: 1 · 잎: 2
```

공식 참고: scikit-learn · DecisionTreeClassifier

한로 강민준

회귀 트리 · 숫자를 구간별 평균으로 예측

```
X=np.array([[100],[200],[300],[700],[800],[900]])
y=np.array([100,120,140,700,720,740])
tree=DecisionTreeRegressor(max_depth=1,random_state=42)
tree.fit(X,y)
print('임계값:',tree.tree_.threshold[0])
print('예측:',tree.predict([[250],[850],[1200]]).tolist())
print('왼쪽 / 오른쪽 평균:',y[:3].mean(),y[3:].mean())
```

E02 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

DecisionTreeRegressor
연속값을 예측하는 트리

squared_error 기준
잎의 y 평균으로 예측

1200은 학습 범위 밖
선형 외삽을 하지 않음

실행 결과 · 요약

```
임계값: 500.0
250 → 120.0
850 → 720.0
1200 → 720.0
```

공식 참고: scikit-learn · DecisionTreeRegressor

3분 퀴즈 · 트리의 예측 원리

Q1. 트리의 질문 임계값은 predict할 때마다 다시 학습할까?

정답·해설 아니다. fit에서 찾은 조건을 예측 때 재사용한다.

Q2. 분류 트리와 회귀 트리의 잎 출력은 같은가?

정답·해설 분류는 클래스·확률, 이 회귀 예제는 잎의 y 평균이다.

Q3. 한 잎에 실패만 있으면 실패 확률 1을 믿어도 될까?

정답·해설 그 잎의 학습 비율이다. 새 입력의 결과를 보장하지 않는다.

공식 참고: scikit-learn · DecisionTreeClassifier / DecisionTreeRegressor

실습 01 · 질문 경로와 잎의 예측

기본필수	E01의 6행 자료로 트리를 학습한다. 250·850의 경로와 임계값 500, 예측 [0, 1]을 직접 설명한다.
응용	E02의 회귀를 실행한다. 왼쪽 평균 120·오른쪽 720을 계산하고, 입력 1200의 예측을 확인한다.
심화	학습점 한 개의 정답을 바꾸어 조건이 어떻게 달라지는지 비교한다. 학습 범위 밖 예측의 한계를 기록한다.

완료 기준: 입력 표 + 분기 경로 + 분류/회귀 예측 + 학습과 추론의 차이

한로 강민준

더 복잡한 트리가 항상 더 좋은 것은 아니다

깊은 트리는 학습 자료의 작은 차이와 우연한 잡음까지 나눌 수 있다.

train 점수만 높고 새로운 자료의 점수가 낮다면 과적합을 의심한다.

깊이·잎 크기 같은 복잡도 제약은 학습 전에 정하는 설정이다.

진단은 같은 분할과 지표로 한다. valid 자료를 맞추려고 임의 삭제하지 않는다.

train–valid 차이는 단서다. 표본이 작다는 이유·데이터 분포 차이도 함께 살핀다.

공식 참고: scikit-learn · Decision Trees / Common pitfalls

트리에서도 결측·범주형 입력 계약은 유지

경로	오늘의 처리	이유
수치형 2열	중앙값 대체 → 스케일링 없음	거리 기반 모델이 아니므로 단위 표준화는 필수 아님
범주형 2열	최빈값 대체 → One-hot	공통 sklearn 경로에 숫자 배열을 전달
결과	전체 train 변환: (60, 8)	수치 2개 + 범주 더미 6개
학습 범위	각 학습 부분에서만 fit	valid·fold 평가 자료는 transform만 수행

일부 구현은 결측을 직접 처리한다. 오늘은 모델 비교를 위해 명시적인 대체 경로를 고정한다.

공식 참고: Pipeline / DecisionTreeClassifier / XGBoost · Parameters

한로 강민준

공통 100건의 분류 트리 연결

```
model=build(DecisionTreeClassifier(
    max_depth=3,min_samples_leaf=3,random_state=42))
model.fit(X_train,y_train)
score=cls_metrics(y_valid,model.predict(X_valid))
est=model['model']
train_f1=f1_score(y_train,model.predict(X_train))
print('shape:',model['pre'].transform(X_train).shape)
print('깊이 / 잎:',est.get_depth(),est.get_n_leaves())
print('train F1:',round(train_f1,3))
print('valid:',score)
```

E03 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

build(estimator)
전처리와 모델을 묶는 함수

min_samples_leaf=3
잎에 최소 3개 학습 행

예측 전처리
train에서 학습한 규칙 재사용

실행 결과 · 요약

```
변환: (60, 8)
깊이 3 · 잎 4
train F1: 0.710
valid F1: 0.333
FP=4 · FN=4
```

공식 참고: scikit-learn · DecisionTreeClassifier / Pipeline

한로 강민준

트리의 복잡도를 줄이는 설정

설정	역할	주의점
max_depth	최대 질문 깊이 제한	너무 작으면 필요한 규칙도 못 배움
min_samples_leaf	각 잎의 최소 학습 행 수	값을 키우면 작은 조각의 분할을 억제
min_samples_split	노드를 더 나눌 최소 학습 행 수	각 자식 잎의 크기와 같은 조건이 아님
ccp_alpha	복잡도 비용을 고려한 사후 가지치기	값이 클수록 더 단순한 후보를 선호

오늘의 기본 비교 트리: max_depth=3, min_samples_leaf=3, random_state=42.

공식 참고: scikit-learn · DecisionTreeClassifier / Cost-complexity pruning

한로 강민준

깊이별 train·valid F1을 함께 기록

```
rows=[]
for depth in [1,2,3,5,None]:
    m=build(DecisionTreeClassifier(max_depth=depth,
        min_samples_leaf=1,random_state=42)).fit(X_train,y_train)
    rows.append({'depth':str(depth),
        'leaves':m['model'].get_n_leaves(),
        'train_F1':f1_score(y_train,m.predict(X_train)),
        'valid_F1':f1_score(y_valid,m.predict(X_valid))})
depth_table=pd.DataFrame(rows)
print(depth_table.round(3).to_string(index=False))
```

E04 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

max_depth 후보
1 · 2 · 3 · 5 · None

다른 조건은 고정
min_samples_leaf=1

기본 비교 Tree의
leaf=3과 다른 진단 실험

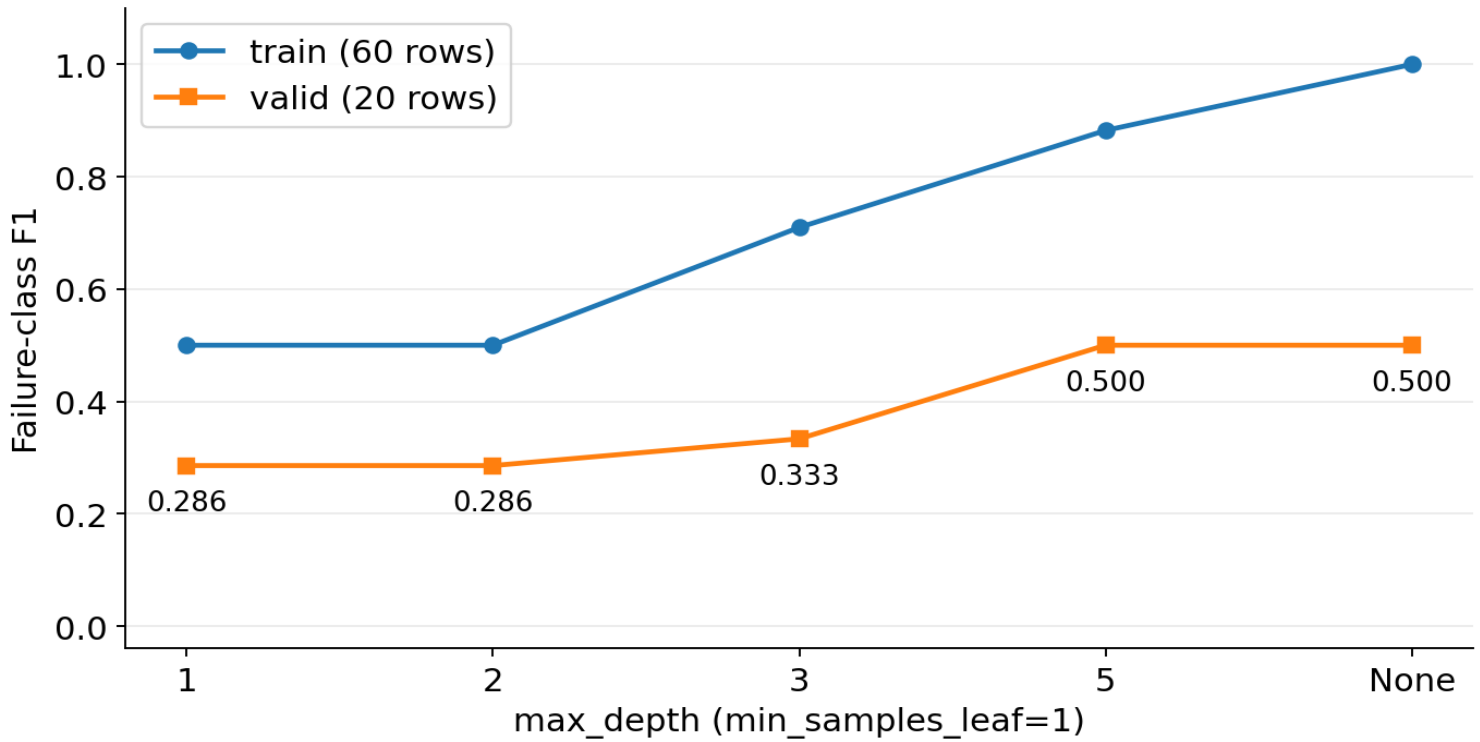
실행 결과 · 요약

```
깊이   train / valid
1     .500 / .286
3     .710 / .333
5     .882 / .500
None 1.000 / .500
```

공식 참고: scikit-learn · DecisionTreeClassifier

한로 강민준

학습 점수 1.000과 검증 점수는 다르다



학습 결과 읽기

제한 없는 트리는
train F1=1.000
valid F1=0.500

깊이 5에서 더 키워도
이번 valid는 개선 없음

한 번의 분할만으로
최적 깊이를 단정하지 않기

이 그림은 깊이별 실험 결과다. epoch에 따른 학습곡선이나 미래 성능 보장이 아니다.

공식 참고: scikit-learn · DecisionTreeClassifier / Cross-validation

3분 퀴즈 · 복잡도와 과적합

Q1. train F1=1이면 모델 선택을 끝내도 될까?

정답·해설 아니다. 검증 점수와 오류를 함께 확인해야 한다.

Q2. min_samples_leaf=5는 “전체 잎 5개”라는 뜻일까?

정답·해설 아니다. 각 잎에 필요한 학습 행의 최소 개수다.

Q3. valid의 잘못 맞힌 행만 지우면 성능 개선인가?

정답·해설 아니다. 평가 자료를 결과에 맞춰 바꾸는 부당한 비교다.

공식 참고: scikit-learn · DecisionTreeClassifier / Common pitfalls

실습 02 · 깊이와 과적합 진단

기본필수	고정 train·valid로 깊이 1·2·3·5·None의 F1을 비교한다. 두 점수를 표로 기록하고 차이를 한 문장으로 해석한다.
응용	같은 깊이에서 min_samples_leaf=1·3·5를 비교한다. 바꾼 조건과 잎 개수를 함께 기록한다.
심화	정상·과소적합·과적합의 후보를 골라 근거를 적는다. 미리 정한 후보만 비교하고 소표본의 한계를 설명한다.

완료 기준: 고정 조건 + 후보별 train/valid 표 + 과적합 진단 + test 미사용

한로 강민준

Random Forest · 다른 트리들을 함께 사용



같은 오류만 반복하는 트리보다 서로 다른 관점을 갖는 트리를 묶는 것이 핵심이다.

공식 참고: scikit-learn · Ensemble methods / RandomForestClassifier

분류 숲의 결합 · 확률 평균과 다수결 구분

원리 예시	계산	결과
가상 트리 3개	실패 확률: 0.51 · 0.51 · 0.01	개별 클래스: 1 · 1 · 0
클래스만 다수결	실패라고 말한 트리가 2개	실패(1)
확률을 먼저 평균	(0.51 + 0.51 + 0.01) / 3 = 0.343	성공(0)
sklearn Random Forest	트리별 클래스 확률을 평균한 뒤 선택	개별 클래스 다수결과 항상 같지 않음

회귀 숲은 각 트리의 숫자 예측을 평균한다. 위 확률 3개는 설명용 가상 값이다.

공식 참고: scikit-learn · RandomForestClassifier / RandomForestRegressor

한로 강민준

Random Forest에서 먼저 볼 설정

설정	수업 값	의미
n_estimators	100	숲에 포함하는 트리 수
max_depth	4	개별 트리의 최대 깊이
min_samples_leaf	2	각 잎의 최소 학습 행 수
max_features	분류 sqrt / 회귀 1.0	분할마다 살펴볼 특성의 후보 수·비율
random_state / n_jobs	42 / 1	난수 재현 / 병렬 작업 수

트리 수를 늘리는 것과 한 트리를 깊게 만드는 것은 서로 다른 조절이다.

공식 참고: scikit-learn · RandomForestClassifier / RandomForestRegressor

Random Forest 분류 · 숲을 Pipeline에 연결

```
rf=RandomForestClassifier(n_estimators=100,max_depth=4,
    min_samples_leaf=2,max_features='sqrt',
    random_state=42,n_jobs=1)
forest=build(rf).fit(X_train,y_train)
score=cls_metrics(y_valid,forest.predict(X_valid))
train_f1=f1_score(y_train,forest.predict(X_train))
print('트리 수:',len(forest['model'].estimators_))
print('train F1:',round(train_f1,3))
print('valid F1:',round(score['F1'],3))
print('FP / FN:',score['FP'],score['FN'])
```

E05 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

RandomForestClassifier
다양한 분류 트리 결합

max_features="sqrt"
매 분할의 특성 후보 제한

estimators_
학습한 개별 트리 목록

실행 결과 · 요약

```
트리 수: 100
train F1: 0.800
valid F1: 0.222
FP=2 · FN=5
```

공식 참고: scikit-learn · RandomForestClassifier

한로 강민준

Random Forest 회귀 · 지표의 단위 확인

```
rf=RandomForestRegressor(n_estimators=100,max_depth=4,
    min_samples_leaf=2,max_features=1.0,random_state=42,n_jobs=1)
forest=build(rf).fit(X_train,yr_train)
score=reg_metrics(yr_valid,forest.predict(X_valid))
print('MAE:',round(score['MAE_ms'],2),'ms')
print('RMSE:',round(score['RMSE_ms'],2),'ms')
print('R2:',round(score['R2'],3))
```

E06 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

RandomForestRegressor
트리별 수치 예측의 평균

max_features=1.0
전체 특성을 후보로 사용

RMSE / MAE
둘 다 정답과 같은 ms 단위

실행 결과 · 요약

```
MAE: 600.34 ms
RMSE: 690.37 ms
R2: -0.276
```

공식 참고: scikit-learn · RandomForestRegressor / Metrics and scoring

한로 강민준

앙상블이 항상 검증 점수를 높이지는 않는다

단일 Tree · 이번 분류

깊이 3 · 잎 최소 3
train F1 = 0.710
valid F1 = 0.333

규칙을 직접 따라보기 쉬움

Forest · 이번 분류

트리 100 · 깊이 4 · 잎 최소 2
train F1 = 0.800
valid F1 = 0.222

더 복잡하지만 이번 결과는 낮음

숲의 이름이나 트리 수가 아니라, 같은 평가 조건의 결과와 오류를 근거로 판단한다.

공식 참고: scikit-learn · Ensemble methods

3분 퀴즈 · 숲의 동작

Q1. Random Forest의 다음 트리는 앞 트리의 잔차를 배우는가?

정답·해설 아니다. 서로 다른 자료·특성 선택을 이용해 트리를 구성한다.

Q2. n_estimators=100은 트리 깊이 100인가?

정답·해설 아니다. 트리 개수다. 깊이는 max_depth 등으로 제한한다.

Q3. 분류 숲은 개별 클래스의 단순 다수결과 항상 같은가?

정답·해설 sklearn은 확률을 평균하므로 다를 수 있다.

공식 참고: scikit-learn · RandomForestClassifier / Ensemble methods

한로 강민준

실습 03 · Random Forest로 결과를 결합

기본필수	트리 100개인 분류 숲을 학습하고 train·valid F1과 FP·FN을 기록한다. 단일 Tree와 비교한다.
응용	회귀 숲을 연결하여 MAE·RMSE를 계산한다. 트리 수 20·100을 비교하되 깊이·분할은 고정한다.
심화	개별 트리의 확률 평균과 숲의 predict_proba를 대조한다. 트리 수 증가가 성능 개선을 보장하지 않는 이유를 적는다.

완료 기준: 고정 설정 + 분류 또는 회귀 비교표 + 결합 원리 + 오류 해석

한로 강민준

특성 중요도 · 모델이 무엇을 사용했는가?

입력 4개 중 모델의 예측에 더 크게 관여한 정보가 무엇인지 살핀다.

학습 때 불순도를 줄인 정도와, 열을 섞었을 때의 점수 감소는 다른 관점이다.

중요도는 모델·데이터·평가지표가 바뀌면 달라질 수 있다.

중요하다는 말은 원인이라는 말이 아니다. 행동의 방향·효과도 따로 검토한다.

현재 Forest의 valid F1은 0.222다. 낮은 예측 성능의 중요도를 업무 우선순위로 단정하지 않는다.

공식 참고: scikit-learn · Permutation importance / Forest feature importances

불순도 기반 중요도 · 학습에서의 기여

계산의 직관

각 분기에서 줄어든 불순도
× 그 분기가 다루는 자료 비중

같은 특성의 기여를 합산
여러 트리의 결과를 평균

읽는 방법

feature_importances_ 사용
값이 클수록 학습 분할에 기여

이 모델은 8개 변환 열
이름·중요도의 순서를 맞추기

이 숲에서는 중요도 합이 1이다. 분기가 전혀 없는 모델은 모두 0일 수 있다.

공식 참고: scikit-learn · Forest feature importances / RandomForestClassifier

한로 강민준

변환된 열 이름과 중요도를 대응시킨다

```
forest=build(new_classifier('Forest')).fit(X_train,y_train)
names=forest['pre'].get_feature_names_out()
mdi=pd.Series(forest['model'].feature_importances_,index=names)
print(mdi.sort_values(ascending=False).round(4).to_string())
print('합:',round(mdi.sum(),4))
```

E07 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

get_feature_names_out()
전처리 뒤 실제 열 순서

feature_importances_
학습 불순도 기반 기여

Series(index=names)
이름과 값을 같은 순서로 연결

실행 결과 · 요약

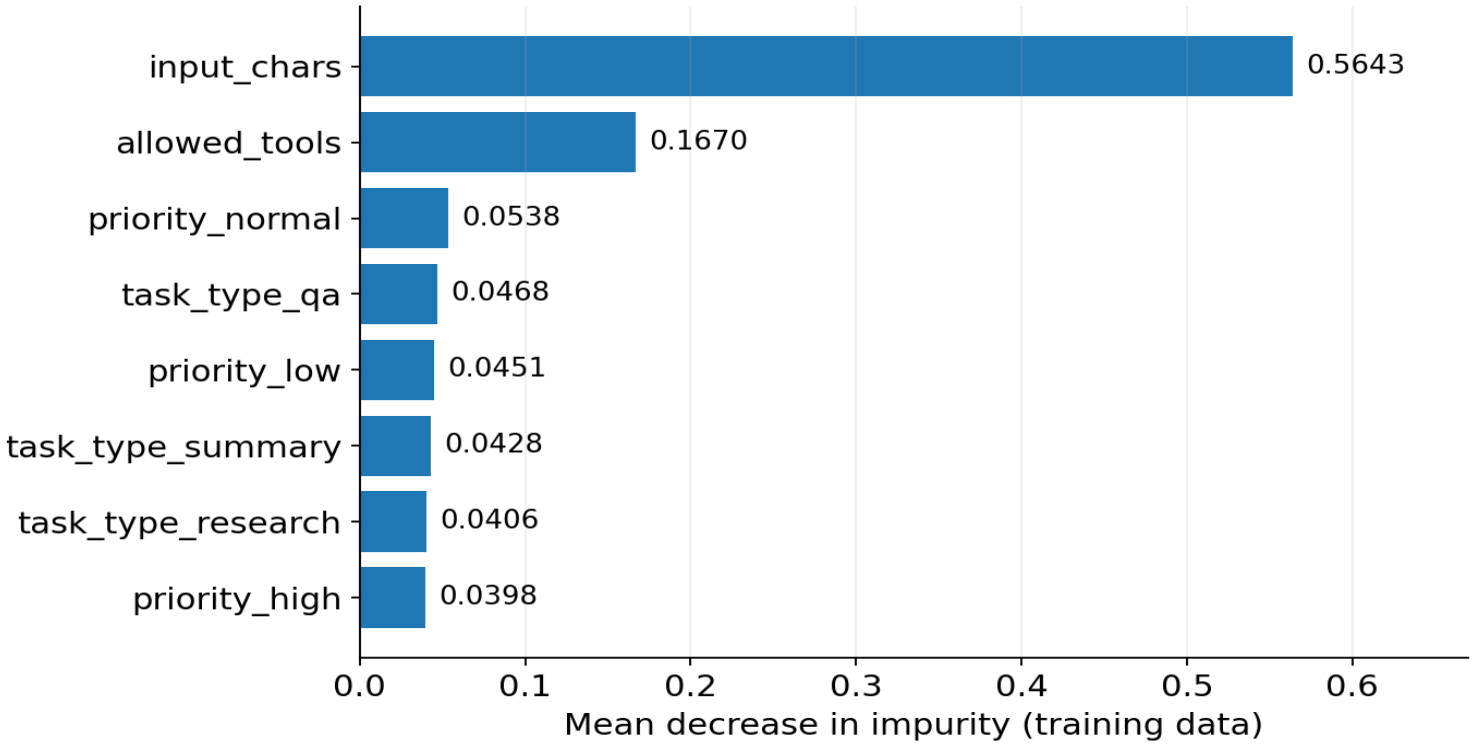
```
상위 두 특성
input_chars:    .5643
allowed_tools: .1670

중요도 합: 1.0000
```

공식 참고: scikit-learn · RandomForestClassifier / Forest feature importances

한로 강민준

8개 변환 특성의 중요도 · 숫자를 읽기



이번 학습에서

input_chars가
가장 큰 MDI를 가짐

범주형 2개는
더미 6개로 나뉨

이 순위는 학습 분할의
기여이지 인과 순위 아님

MDI는 학습 자료에서 계산한다. ID·실행 후 정보가 섞이지 않았는지 먼저 감사한다.

공식 참고: scikit-learn · Forest feature importances

한로 강민준

중요도를 해석할 때 피해야 할 결론

값이 다양한 특성은 분할 후보도 많아 불순도 중요도가 높게 보일 수 있다.

상관된 입력끼리는 예측에 필요한 정보가 중복되어 기여가 나뉠 수 있다.

중요도는 양·음의 영향 방향이나 실제 원인 관계를 알려 주지 않는다.

중요도 순위를 본 뒤 평가셋에 맞춰 열을 반복 삭제하면 선택 편향이 커진다.

열 선택이 필요하면 그 과정도 학습 fold 안에서 수행하고, 변경한 실험으로 따로 기록한다.

공식 참고: scikit-learn · Permutation importance / Forest feature importances / Common pitfalls

한로 강민준

순열 중요도 · 한 열의 연결을 깨뜨려 본다

기준 점수

학습한 모델 고정
valid F1 계산

→

한 열 섞기

원본 열의 값을
행 사이에서 교환

→

점수 차이

기준 F1 – 섞은 F1
여러 번 반복

모델을 다시 fit하지 않는다. 원본 범주 열을 섞으면 해당 더미 묶음을 함께 평가한다.

공식 참고: scikit-learn · permutation_importance

한로 강민준

전체 Pipeline에 원본 네 열을 전달

```
forest=build(new_classifier('Forest')).fit(X_train,y_train)
perm=permutation_importance(forest,X_valid,y_valid,
    scoring='f1',n_repeats=20,random_state=42,n_jobs=1)
importance=pd.DataFrame({'feature':FEATURES,
    'F1_drop_mean':perm.importances_mean,
    'repeat_std':perm.importances_std})
base_f1=f1_score(y_valid,forest.predict(X_valid))
print('기준 F1:',round(base_f1,3))
print(importance.round(4).to_string(index=False))
```

E08 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

scoring="f1"
실패 클래스 F1의 감소량

n_repeats=20
20번 섞은 변동도 기록

평균·표준편차
신뢰구간과 구분

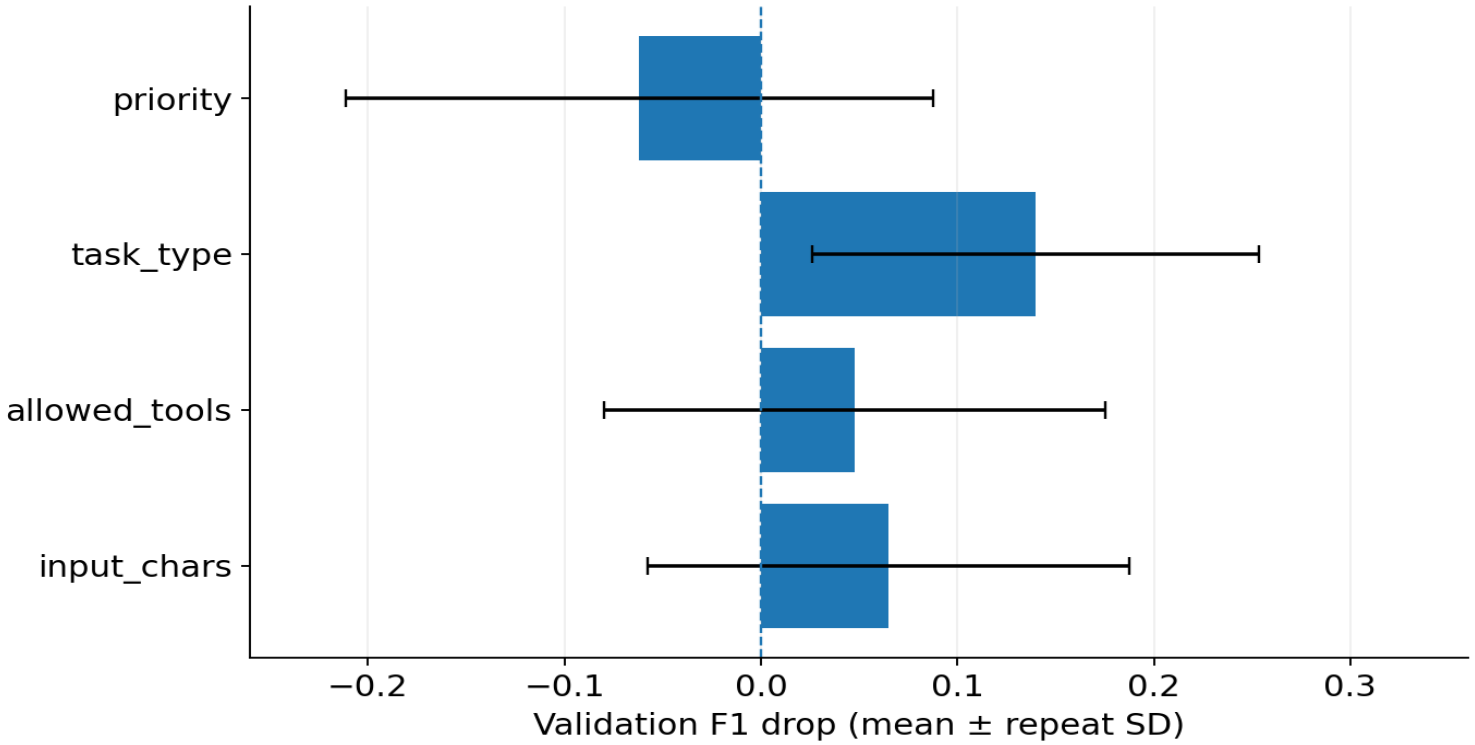
실행 결과 · 요약

```
기준 F1: 0.222
감소량 평균
chars       .0649
tools       .0478
task_type   .1396
priority   -.0619
```

공식 참고: scikit-learn · permutation_importance / Permutation importance

한로 강민준

순열 중요도 · 음수도 관측될 수 있다



평균 ± 반복 표준편차

priority의 평균은 음수
섞은 뒤 점수가 더 높았음

20행의 작은 평가 자료
순위·부호의 불안정성

즉시 삭제할 근거나
인과 효과로 해석하지 않기

중요도 비교는 이번 모델과 F1에 대한 진단이다. 더 좋은 데이터·평가 설계가 먼저다.

공식 참고: scikit-learn · Permutation importance

한로 강민준

3분 퀴즈 · 중요도의 의미

Q1. MDI 0.5는 정확도를 50% 올렸다는 뜻인가?

정답·해설 아니다. 이 모델에서 정규화된 불순도 감소 기여다.

Q2. 순열 중요도는 열을 섞을 때마다 모델을 다시 학습하는가?

정답·해설 아니다. 이미 학습한 모델을 고정하고 점수 변화를 측정한다.

Q3. 음수 중요도가 나오면 그 열을 바로 삭제해야 할까?

정답·해설 아니다. 표본·반복 변동·성능 수준·상관 관계를 먼저 살핀다.

공식 참고: scikit-learn · Forest feature importances / Permutation importance

한로 강민준

실습 04 · 특성 중요도와 해석의 한계

기본필수	Forest의 변환 특성 8개와 MDI 값을 대응시킨다. 상위 특성 2개와 "원인이라고 말할 수 없는 이유"를 적는다.
응용	원본 4개 열에 순열 중요도를 적용한다. 평균·반복 표준편차를 저장하고 MDI와 순위가 다른 이유를 제안한다.
심화	상관된 열·고유값이 많은 열이 영향을 줄 수 있는 상황을 설계한다. 열 추가·선택을 CV 밖에서 하면 안 되는 이유를 적는다.

완료 기준: 이름-값 대응표 + 해석 2문장 + 편향·소표본 한계 + 데이터 누수 점검

Boosting · 현재 예측의 부족한 부분을 보정

처음 예측

간단한 출발점
예: 정답 평균

→

보정 트리

현재 손실을 줄일
방향을 학습

→

예측 갱신

이전 예측 + 학습률 × 보정
순차적으로 반복

잔차를 그대로 배우는 설명은 제곱오차의 직관이다. 분류는 손실의 기울기 정보를 사용한다.

공식 참고: XGBoost · Boosted trees / Ensemble methods

한로 강민준

손계산 예제 · 잔차 보정 한 번의 효과

```
X=np.array([[1],[2],[3],[4]])
y=np.array([100.,120.,200.,220.])
initial=np.full(4,y.mean())
residual=y-initial
correction=DecisionTreeRegressor(
    max_depth=1,random_state=42).fit(X,residual)
updated=initial+0.5*correction.predict(X)
print(updated.tolist())
print(np.mean((y-initial)**2),np.mean((y-updated)**2))
```

E19 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

residual = y − 예측
제곱오차에서의 보정 목표

학습률 0.5
보정량을 절반만 반영

설명용 작은 예제
XGBoost 내부 식과 구분

실행 결과 · 요약

처음: 모두 160
보정 후:
135,135,185,185

MSE 2600 → 725

공식 참고: XGBoost · Boosted trees

한로 강민준

Bagging과 Boosting을 구분한다

관점	Random Forest	XGBoost · 트리 부스팅
학습 관계	여러 트리를 독립적으로 구성	현재 예측을 보정하며 순차 추가
다양성·보정	표본과 특성 후보의 무작위성	손실의 기울기·곡률에 따른 보정
예측 결합	확률 또는 수치의 평균	기본 점수에 보정값을 누적
핵심 조절	트리 수·깊이·잎 크기·특성 수	깊이·학습률·회차·표본/특성 비율

둘 다 여러 트리를 사용하지만, 학습 순서와 결합 방식이 다르다.

공식 참고: Ensemble methods / XGBoost · Boosted trees

한로 강민준

XGBoost · 규제를 포함한 트리 부스팅 구현

손실을 줄이는 여러 트리를 순서대로 추가하며 예측을 개선한다.

트리 복잡도와 잎 값에 대한 규제를 사용해 지나친 적합을 제어한다.

sklearn처럼 fit·predict를 사용하고, 기존 전처리 Pipeline에 연결할 수 있다.

오늘은 hist 방식과 CPU를 사용한다. GPU·CUDA 설치는 선행 조건이 아니다.

학습률을 낮추거나 규제를 강화해도 항상 성능이 좋아지는 것은 아니다. 비교로 확인한다.

공식 참고: XGBoost · sklearn estimator / XGBoost · Parameters

XGBoost의 주요 설정 · 역할을 먼저 이해

설정	수업 값	역할
n_estimators / learning_rate	80 / 0.05	보정 회차 / 한 회의 보정 반영 정도
max_depth	2	각 보정 트리의 최대 깊이
min_child_weight	1	자식 노드에 필요한 Hessian 합의 하한
subsample / colsample_bytree	0.8 / 1.0	행 / 트리별 특성의 사용 비율
reg_lambda	1.0	잎 가중치에 대한 L2 규제
tree_method / device / n_jobs	hist / cpu / 1	구축 방식 / 장치 / 병렬 작업 수

min_child_weight는 min_samples_leaf와 다르다. 분류에서 “최소 행 수”로 해석하지 않는다.

공식 참고: XGBoost · Parameters

한로 강민준

XGBoost 분류 · 실패 확률 모델 연결

```
xgb=XGBClassifier(n_estimators=80,max_depth=2,
    learning_rate=0.05,subsample=0.8,
    colsample_bytree=1.0,min_child_weight=1,
    reg_lambda=1.0,objective='binary:logistic',
    eval_metric='logloss',tree_method='hist',
    device='cpu',random_state=42,n_jobs=1)
model=build(xgb).fit(X_train,y_train)
print(cls_metrics(y_valid,model.predict(X_valid)))
```

E09 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

objective
binary:logistic = 이진 분류

eval_metric="logloss"
학습 진단용 손실

고정 80회차 모델
조기 종료 예제와 구분

실행 결과 · 요약

```
valid F1: 0.182
Precision: 0.200
Recall: 0.167
FP=4 · FN=5

F1과 logloss는 다름
```

공식 참고: XGBoost 3.1 · Python API / XGBoost · Parameters

XGBoost 회귀 · 숫자 정답으로 변경

```
xgb=XGBRegressor(n_estimators=80,max_depth=2,
    learning_rate=0.05,subsample=0.8,
    colsample_bytree=1.0,min_child_weight=1,
    reg_lambda=1.0,objective='reg:squarederror',
    eval_metric='rmse',tree_method='hist',
    device='cpu',random_state=42,n_jobs=1)
model=build(xgb).fit(X_train,yr_train)
print(reg_metrics(yr_valid,model.predict(X_valid)))
```

E10 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

objective
reg:squarederror = 제곱오차

X와 분할 ID는 동일
y만 elapsed_ms로 변경

회귀 비교는
분류 F1과 합치지 않음

실행 결과 · 요약

```
MAE: 609.34 ms
RMSE: 700.02 ms
R2: -0.312
```

공식 참고: XGBoost 3.1 · Python API / Metrics and scoring

한로 강민준

3분 퀴즈 · XGBoost의 설정

Q1. learning_rate를 줄이면 같은 회차에서도 반드시 개선될까?

정답·해설 아니다. 보정이 부족해질 수 있어 회차·복잡도와 함께 비교한다.

Q2. binary:logistic의 학습 진단 logloss는 F1과 같은가?

정답·해설 아니다. 손실과 최종 선택 지표의 역할을 구분한다.

Q3. min_child_weight=5는 항상 자식 행 5개 이상인가?

정답·해설 아니다. Hessian 합의 조건이며 행 개수와 일반적으로 다르다.

공식 참고: XGBoost · Parameters / XGBoost 3.1 · Python API

실습 05 · XGBoost 구현과 손실·지표 구분

기본필수	고정 80회차·깊이 2의 분류 모델을 Pipeline으로 학습한다. train·valid F1과 FP·FN을 기록한다.
응용	같은 입력·분할로 회귀 모델을 학습한다. objective와 정답을 바꾸고 MAE·RMSE를 비교한다.
심화	미리 정한 학습률·회차 조합 2개만 비교한다. 고정 조건과 진단 지표·선택 지표를 따로 기록한다.

완료 기준: 모델 설정 + train/valid 결과 + 목적함수·지표 설명 + test 미사용

한로 강민준

과적합 제어 · 구조와 학습량을 함께 점검

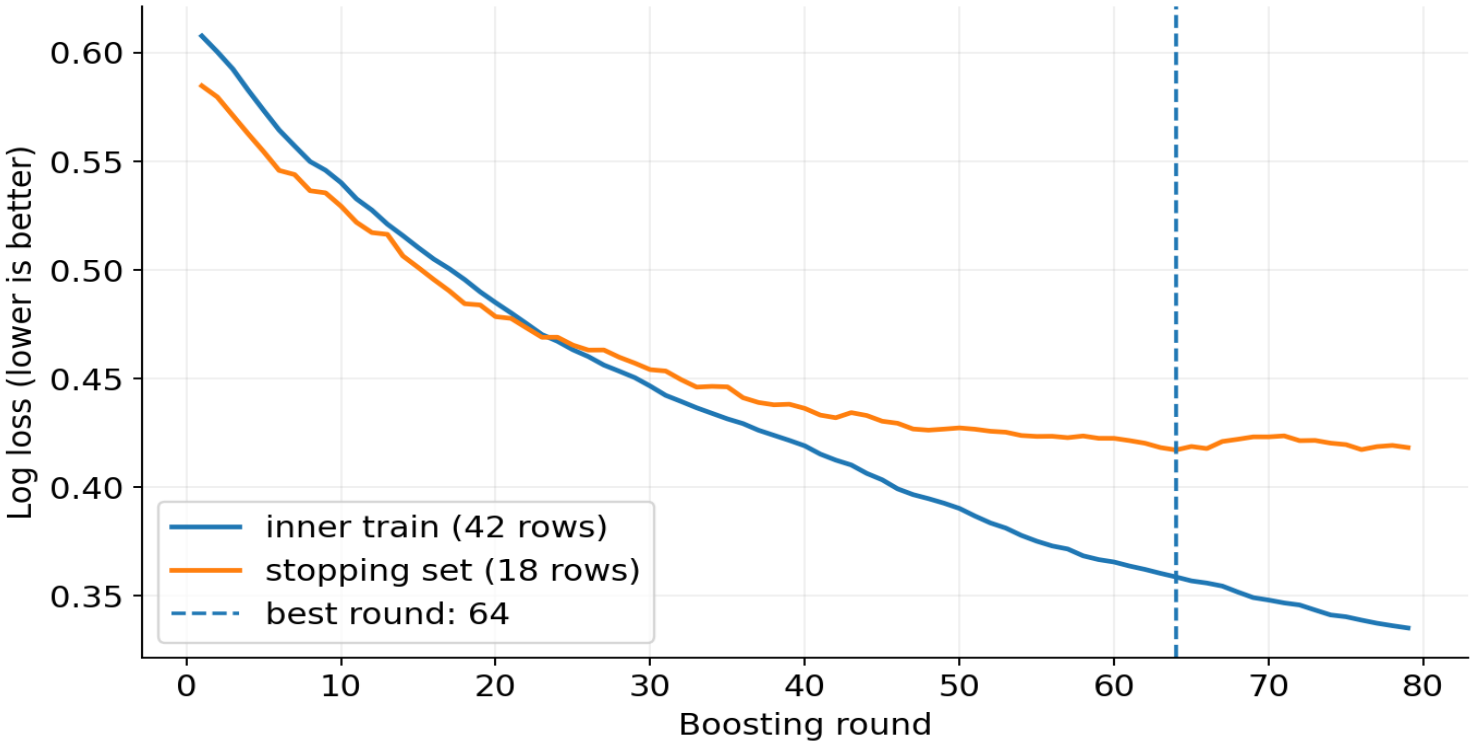
모델	제어하는 것	진단할 증거
Decision Tree	깊이·잎 크기·가지치기	train/valid 격차 · 잎 개수
Random Forest	개별 트리 복잡도·특성 후보·트리 수	반복 결과 · 계산량 · fold별 편차
XGBoost	깊이·규제·학습률·회차·부분 표본	손실 변화 · stop 자료 · 과도한 학습량
공통 원칙	후보·데이터·지표를 미리 고정	한 번에 바꾼 조건 · 실패 원인 · test 보관

규제와 단순화는 무조건 강하게 하는 것이 아니다. 과소적합과의 균형을 검증한다.

공식 참고: DecisionTreeClassifier / RandomForestClassifier / XGBoost · Parameters

한로 강민준

조기 종료 · 손실 개선이 멈추는 지점을 찾기



독립 시연 결과

train 60 내부에서
inner 42 / stop 18

최적 회차: 64
실제 학습: 79회
15회 개선 없으면 종료

기본 비교의 XGB80과
별도 실험으로 기록

stop 자료도 모델 선택에 사용된다. 오늘의 valid20·test20을 여기에 넣지 않는다.

공식 참고: XGBoost · sklearn estimator / XGBoost 3.1 · Python API

교차검증 · 한 번의 검증 분할에 덜 의존

train 60만 사용

5개의 부분으로 나눔
valid·test와 구별

→

5번 반복

48개로 학습
남은 12개로 평가

→

결과 비교

fold 점수 · 평균 · 편차
후보마다 같은 분할

CV도 후보 선택에 사용한 결과다. 평균이 높다는 사실만으로 최종 성능을 확정하지 않는다.

공식 참고: scikit-learn · Cross-validation / StratifiedKFold

5-fold의 안쪽에서 전처리도 다시 학습

train 60건을 12건씩 5개로 구성 · 진한 칸은 해당 회차의 fold 평가 자료

회차 1	**평가 12**	학습 12	학습 12	학습 12	학습 12
회차 2	학습 12	**평가 12**	학습 12	학습 12	학습 12
회차 3	학습 12	학습 12	**평가 12**	학습 12	학습 12
회차 4	학습 12	학습 12	학습 12	**평가 12**	학습 12
회차 5	학습 12	학습 12	학습 12	학습 12	**평가 12**

각 회차: 전처리 fit(48) → 모델 fit(48) → transform·predict(12)

cross_validate의 test_score는 “fold 평가 점수”라는 반환 이름이다. 보관한 test20이 아니다.

공식 참고: scikit-learn · StratifiedKFold / Cross-validation

한로 강민준

틀린 CV와 올바른 CV · 학습 범위가 다르다

잘못된 순서

train60 전체에 전처리 fit
변환된 배열을 5-fold로 나눔

fold 평가 행의 통계가
학습 전처리에 이미 들어감

올바른 순서

아직 fit하지 않은 Pipeline 전달
fold 학습48에서 전처리 fit
같은 48로 모델 fit
fold 평가12는 transform·predict

train60 안에서도 학습48과 fold 평가12는 분리해야 한다.

공식 참고: scikit-learn · Common pitfalls / Pipeline / cross_validate

한로 강민준

cross_validate에 Pipeline 전체를 전달

```
model=build(new_classifier('Forest'))
cv_result=cross_validate(model,X_train,y_train,cv=CLS_CV,
    scoring='f1',return_train_score=True,return_estimator=True,
    n_jobs=1,error_score='raise')
print('fold F1:',np.round(cv_result['test_score'],3).tolist())
print('평균:',round(cv_result['test_score'].mean(),3))
print('표준편차:',round(cv_result['test_score'].std(ddof=0),3))
print('train 평균:',round(cv_result['train_score'].mean(),3))
```

E13 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

CLS_CV
고정한 분류용 5-fold 인덱스

return_train_score=True
학습과 평가 점수 함께 반환

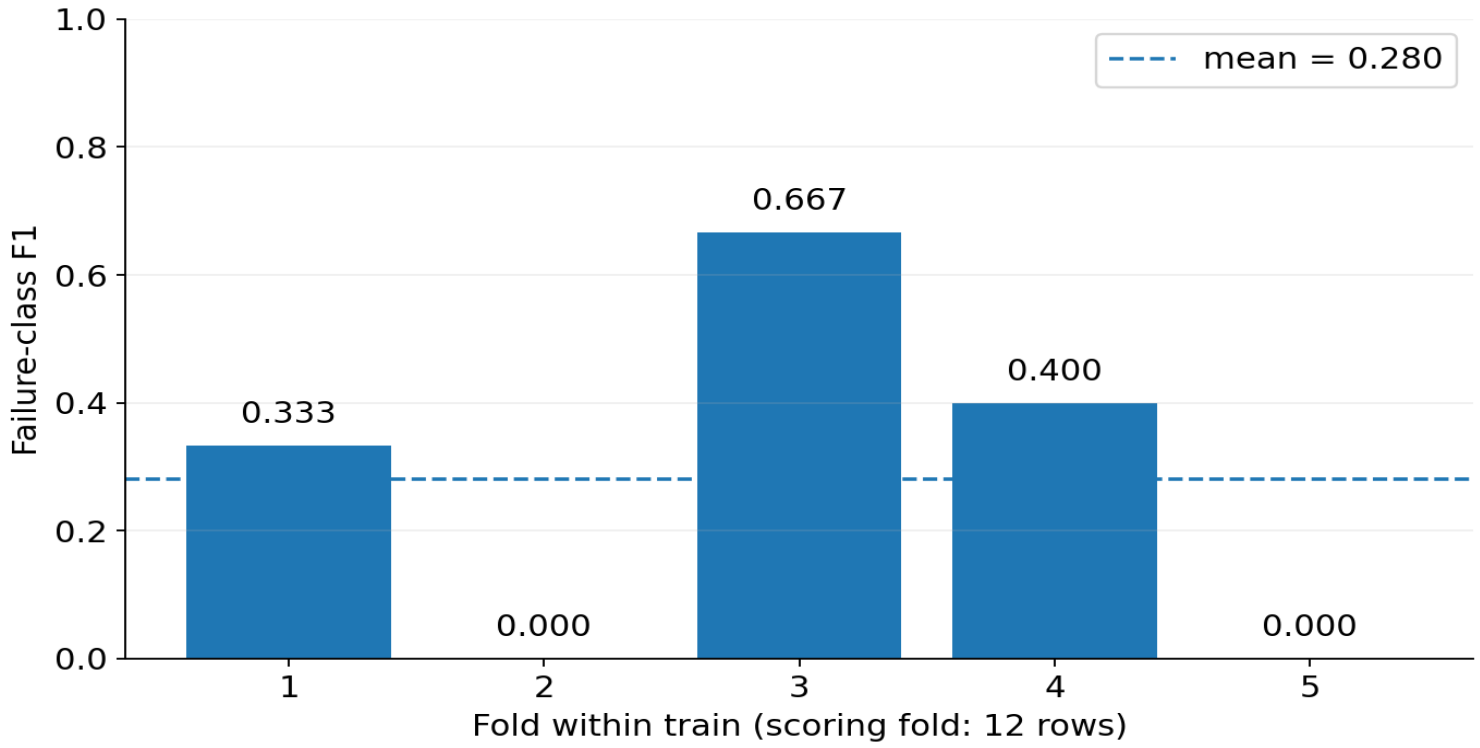
std(ddof=0)
5개 점수의 기술적 편차

실행 결과 · 요약

```
fold F1:
.333,.000,.667,
.400,.000
평균 .280
표준편차 .254
```

공식 참고: scikit-learn · cross_validate / StratifiedKFold

fold별 결과 · 평균 하나로 가리지 않기



Forest · 실패 F1

fold 2·5는 F1=0
fold 3은 F1 약 0.667

평균 0.280
표준편차 0.254

평가 자료 12건의
구성에 민감한 결과

fold 표준편차는 95% 신뢰구간이 아니다. “정확한 기대 성능 범위”로 발표하지 않는다.

공식 참고: scikit-learn · Cross-validation / StratifiedKFold

각 fold의 중앙값이 학습48에서 왔는지 검사

```
cv=cross_validate(build(new_classifier('Tree')),
    X_train,y_train,cv=CLS_CV,scoring='f1',
    return_estimator=True,error_score='raise')
for (a,b),m in zip(CLS_CV,cv['estimator']):
    num=m['pre'].named_transformers_['num']
    actual=num['imputer'].statistics_
    expected=X_train.iloc[a][NUM].median()
    assert np.allclose(actual,expected)
    assert not set(a)&set(b)
print('전 fold 중앙값·분할 검사 통과')
```

E16 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

return_estimator=True
fold별 학습 모델을 받기

.iloc[fit_idx]
그 fold의 학습 행만 선택

np.allclose()
계산한 수치의 근사 일치 확인

실행 결과 · 요약

```
fold별 검사
True × 5

학습 48 / 평가 12
교집합: 0
```

공식 참고: scikit-learn · cross_validate / Common pitfalls

한로 강민준

문제에 맞는 분할기와 점수 해석

상황	오늘의 선택	주의할 점
분류 · 독립 요청	StratifiedKFold(5, shuffle=True)	실패 비율 유지. 요청 복제본은 따로 검사
회귀 · 독립 요청	KFold(5, shuffle=True)	연속 정답을 그대로 stratify하지 않음
같은 요청의 여러 단계	그룹 단위 분할을 별도 설계	request_id가 학습·평가를 가로지르지 않게
미래 요청 예측	시간 순서 분할을 별도 설계	미래 정보·통계가 학습으로 들어가지 않게

동일 문제의 후보들은 같은 fold 인덱스로 비교한다. 회귀와 분류의 지표는 합산하지 않는다.

공식 참고: scikit-learn · Cross-validation / StratifiedKFold / KFold

한로 강민준

3분 퀴즈 · fold 안의 학습 범위

Q1. 전처리만 먼저 train60으로 fit해 놓고 CV해도 될까?

정답·해설 안 된다. 각 fold의 학습48 안에서 다시 fit해야 한다.

Q2. cross_validate의 test_score를 출력하면 보관 test를 쓴 것인가?

정답·해설 아니다. 오늘은 train60 내부의 fold 평가 점수다.

Q3. 평균 F1 ± 표준편차를 95% 신뢰구간이라고 해도 될까?

정답·해설 아니다. fold 결과의 산포를 기록한 값이다.

공식 참고: scikit-learn · cross_validate / Common pitfalls

한로 강민준

실습 06 · train 내부 교차검증 적용

기본필수	Tree·Forest·XGB의 Pipeline에 같은 5-fold를 적용한다. 분류 F1의 fold 점수·평균·표준편차를 기록한다.
응용	회귀 KFold와 음수 RMSE scorer를 적용한다. 부호를 되돌리고 fold별 RMSE의 평균을 비교한다.
심화	각 fold 전처리의 중앙값 출처를 검사한다. GridSearchCV 4후보 또는 내부 조기 종료를 별도 실험한다.

완료 기준: fold ID + 후보별 점수·편차 + fold 안에서 fit한 증거 + test 보관

최종 비교의 조건 · 바뀐 것과 유지한 것

항목	고정 내용	비교 시 주의
자료·입력	동일 100건·입력4열·분할 ID	원본·분할 내 행 순서까지 유지
전처리	이전 선형/KNN: 수치 standard 트리/숲/XGB: 수치 스케일링 없음	모델에 맞춘 학습 절차 전체를 비교
후보	이전 3개 + Tree·Forest·XGB	깊이·회차는 사전 고정한 설정
오늘의 잠정 기준	분류 CV 평균 F1 최대 회귀 CV 평균 RMSE 최소	valid는 진단, test는 최종 확인 전 보관

동점이면 더 단순한 후보를 우선한다. 작은 차이는 절대적 우열이 아니라 추가 검증 대상으로 기록한다.

공식 참고: scikit-learn · Cross-validation / Metrics and scoring

한로 강민준

분류 비교표 · 같은 fold에 같은 후보 순서

```
rows=[]
for name,m in classifier_candidates().items():
    cv=cross_validate(m,X_train,y_train,
        cv=CLS_CV,scoring='f1',n_jobs=1)
    m.fit(X_train,y_train)
    score=cls_metrics(y_valid,m.predict(X_valid))
    rows.append({'model':name,
        'CV_F1':cv['test_score'].mean(),
        'valid_F1':score['F1']})
print(pd.DataFrame(rows).round(3))
```

E14 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

classifier_candidates()
이전 3개 + 새 모델 3개

CV 뒤 fit(train60)
valid 예측을 별도로 기록

CV 평균과 valid 점수
서로 다른 열로 저장

실행 결과 · 요약

```
CV F1 / valid F1
Tree   .542 / .333
Forest .280 / .222
XGB    .500 / .182
```

공식 참고: scikit-learn · cross_validate / Metrics and scoring

분류 결과 · CV와 valid의 순위가 다르다

모델	CV F1 평균 ± 표준편차	valid F1	valid FP / FN
Dummy	0.000 ± 0.000	0.000	0 / 6
Logistic	0.522 ± 0.209	0.333	4 / 4
KNN3	0.336 ± 0.176	0.545	2 / 3
Tree	0.542 ± 0.124	0.333	4 / 4
Forest	0.280 ± 0.254	0.222	2 / 5
XGB	0.500 ± 0.123	0.182	4 / 5

CV 평균은 Tree가 가장 높고 valid는 KNN3이 가장 높다. 유리한 점수만 골라 보고하지 않는다.

공식 참고: scikit-learn · Cross-validation

한로 강민준

회귀 비교표 · 음수 scorer를 RMSE로 변환

```
rows=[]
for name,m in regressor_candidates().items():
    cv=cross_validate(m,X_train,yr_train,cv=REG_CV,
        scoring='neg_root_mean_squared_error')
    m.fit(X_train,yr_train)
    pred=m.predict(X_valid)
    rows.append({'model':name,
        'CV_RMSE':(-cv['test_score']).mean(),
        'valid_RMSE':root_mean_squared_error(yr_valid,pred)})
print(pd.DataFrame(rows).round(2))
```

E15 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

REG_CV
회귀용 KFold 5개

neg_root_mean_squared_error
큰 값이 좋도록 음수 반환

부호를 되돌린 평균
단위는 ms

실행 결과 · 요약

```
CV RMSE (ms)
KNN15: 598.85
Tree:  599.63
Forest:624.01
XGB:   655.79
```

공식 참고: scikit-learn · cross_validate / KFold / Metrics and scoring

회귀 결과 · 0.78ms 차이를 과장하지 않기

모델	CV RMSE 평균 ± 편차 (ms)	valid RMSE	valid MAE
Dummy	606.18 ± 100.84	658.75	541.26
Linear	639.83 ± 57.72	640.00	551.80
KNN15	598.85 ± 91.21	641.79	530.06
Tree	599.63 ± 96.12	719.00	632.03
Forest	624.01 ± 96.97	690.37	600.34
XGB	655.79 ± 92.17	700.02	609.34

CV는 KNN15와 Tree가 매우 가깝다. valid는 Linear가 낮다. 앙상블이 항상 우세하지 않다.

공식 참고: scikit-learn · Cross-validation / Metrics and scoring

한로 강민준

실험 보고서 · 점수보다 판단 과정을 남긴다

질문	분류 예시	회귀 예시
사전 주 지표	실패 클래스의 CV 평균 F1	CV 평균 RMSE · ms
잠정 선택	고정 후보 중 Tree · 0.542	KNN15 · 598.85ms
반대되는 증거	valid 최고는 KNN3 · 0.545	valid RMSE 최고는 Linear · 640.00ms
남은 한계	Tree는 valid 실패 6건 중 4건 놓침	KNN15와 Tree의 CV 차이 약 0.78ms
다음 행동	후보·분할·지표 고정 후 추가 검증	소표본·순위 변동 기록 후 최종 평가 준비

10/26의 valid 기준과 오늘의 CV 기준이 다르다. 판단 기준이 바뀐 사실도 보고서에 남긴다.

공식 참고: scikit-learn · Cross-validation

실습 프로젝트 · 데이터와 실험 결과를 나눈다

```
day7_ensemble/
  main.py
  preprocessing.py
  evaluate.py
  data/requests.csv
  data/split_manifest.csv
  data/split_order.csv
  outputs/
    classification_comparison.csv
    cv_scores.csv
    feature_importance.csv
    validation_predictions.csv
  reports/ensemble_report.md
```

파일보다 먼저 책임을 분리

main: 읽기·실험 실행
preprocessing: 입력 변환 구성
evaluate: 지표·CV·결과 검증

README: 실행법·버전·한계
reports: 선택 근거·오류 설명
outputs: 다시 생성할 수 있는 결과

test 점수 열을 만들지 않는다. “보관·미평가”는 누락이 아니라 이번 수업의 정책이다.

한로 강민준

검사 · 새 범주와 누락 열을 구분

```
model=build(new_classifier('Forest')).fit(X_train,y_train)
imp=model['pre'].named_transformers_['num']['imputer']
before=imp.statistics_.copy()
new=X_valid.iloc[:2].copy()
new['task_type'],new['priority']='translation','urgent'
model.predict(require_features(new))
assert np.array_equal(before,imp.statistics_)
print(model['pre'].transform(new).shape)
# priority를 지운 입력은 require_features에서 차단한다.
```

E17 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

미지 범주
encoder가 기존 8열 유지

필수 열 누락
예측 전에 명확히 차단

학습 통계
새 요청 예측으로 변경 안 됨

실행 결과 · 요약

```
새 범주: (2, 8)
통계 불변: True
누락 열 차단: True
```

공식 참고: scikit-learn · Pipeline / Common pitfalls

저장·복원 · 직접 만든 모델만 읽는다

```
import tempfile,joblib
model=build(new_classifier('Forest')).fit(X_train,y_train)
with tempfile.TemporaryDirectory() as folder:
    path=Path(folder)/'selected_pipeline.joblib'
    joblib.dump(model,path)
    restored=joblib.load(path)  # 자신이 방금 만든 파일만
    same=np.array_equal(model.predict(X_valid),
                        restored.predict(X_valid))
    print('복원 예측 동일:',same)
```

E18 | 독립 실행 전체 코드: 발표자 노트

함수 · 매개변수

joblib.dump()
학습 전처리와 모델 저장

joblib.load()
직접 만든 신뢰하는 파일만

버전·입력 스키마도 기록
다른 환경 호환 보장 아님

실행 결과 · 요약

```
복원 예측 동일:
True

입력 4열 유지
```

공식 참고: scikit-learn · Model persistence

60분 통합 · 이미 만든 결과를 연결한다

실험 구성

입력·분할 계약 확인
사전 고정 모델·설정

→

측정·저장

fold별 CV·valid 결과
중요도·예측 파일

→

설명·검증

오류·한계·선택 근거
README·Git 이력

7교시 실습 25분 + 8교시 실습 35분 · 새 기능보다 한 번에 재현되는 완료를 우선한다.

한로 강민준

실습 07 · 앙상블 비교 프로젝트 통합

기본필수	고정 분할로 Dummy·Tree·Forest·XGB 분류를 연결한다. CV 5-fold·valid 비교표·중요도 표를 한 실행으로 만든다.
응용	전날 모델과 회귀 비교를 추가한다. 모듈을 분리하고 선택 Pipeline의 저장·복원을 대조한다.
심화	fold 전처리 통계·새 범주·필수 열 누락 중 두 가지 이상을 자동 검사하고 별도 탐색 실험의 결과를 기록한다.

완료 기준: 실행 초안 + 분류 비교표 + CV 기록 + 중요도 표 + test 보관

한로 강민준

최종 제출 · 앙상블 모델 실험 보고서

산출물	필수 내용
classification_comparison.csv	모델·설정·CV 평균/편차·valid F1·FP·FN
cv_scores.csv	모델·fold 번호·학습/평가 개수·각 점수
feature_importance.csv	특성 이름·방법(MDI 또는 순열)·값·평가 자료
validation_predictions.csv	request_id·모델·실제값·예측값
reports/ensemble_report.md	선택 기준·근거 3개·오류 사례·해석 한계
README · 버전 · Git 이력	실행 명령·환경·데이터 출처·변경 기록

응용 추가: regression_comparison.csv · 저장/복원 결과 · 더 넓은 실험 기록.

한로 강민준

정상·경계·실패 상황을 따로 검증

상황	기대 결과	판정 근거
정상 valid20	20개 예측·ID 순서 유지	NaN·inf 없이 지표 재계산 가능
수치 결측·미지 범주	기존 규칙으로 8열 변환	중앙값·범주 목록이 변하지 않음
priority 열 누락	명확한 오류 메시지	자동 추측으로 필수 입력을 채우지 않음
CV의 한 fold	학습48·평가12·교집합0	대체값이 fold 학습48의 중앙값
직접 저장한 모델	동일 환경의 예측 일치	기존 결과와 복원 후 결과 대조

오류 없이 끝났다는 것과 요구사항대로 동작했다는 것은 다르다.

공식 참고: scikit-learn · Common pitfalls / Model persistence

형성평가 · 높은 성능보다 올바른 실험

항목	배점	충족 기준
데이터·누수 방지	20	입력 계약·동일 ID·fold 내 전처리·test 보관
모델 구현	25	Dummy·Tree·Forest·XGBoost 분류 실행
과적합·교차검증	25	고정 조건·train/valid 진단·5-fold 점수 기록
중요도·선택 근거	20	특성명 대응·해석 한계·지표·오류 근거
재현·설명·협업	10	README·버전·결과 재생성·Git 변경 이력

기본필수만으로 100점 기준을 충족할 수 있다. 과목 최종 실습평가는 10/28에 진행한다.

실습 08 · 최종 검증·보고서·시연

기본필수	한 번의 실행으로 비교표·CV 기록·중요도를 재생성한다. 선택 이유·오류 사례·한계·test 보관을 설명한다.
응용	짝이 README만 보고 실행하게 한다. 회귀 비교나 복원 예측을 대조하고 수정 전후 차이를 적는다.
심화	정상·경계·실패 테스트를 시연한다. 다음 수업에서도 유지할 분할·특성 선택·최종 평가 정책을 문서화한다.

완료 기준: 코드 + 실험표 + 중요도 + 보고서 + 실행 증거 + Git 이력

한로 강민준

마무리 3분 확인 퀴즈

Q1. Random Forest와 XGBoost의 가장 큰 학습 차이는?

정답·해설 독립적인 트리의 결합과 순차적인 손실 보정의 차이다.

Q2. 중요도 1위 특성이 실패의 원인이라고 말해도 되는가?

정답·해설 아니다. 특정 모델과 데이터에서의 의존성을 진단한 것이다.

Q3. 다음 수업에서 새 특성을 만들면 어디에서 선택해야 하는가?

정답·해설 학습 fold 안에서 선택하고, 최종 test는 선택 이후에 확인한다.

공식 참고: scikit-learn · Ensemble methods / Permutation importance / Common pitfalls

한로 강민준

오늘의 핵심 정리

모델: 트리의 조건 분기, 숲의 결합, 부스팅의 순차 보정을 구분한다.

해석: 중요도의 계산 대상·편향·변동을 설명하고 인과로 과장하지 않는다.

검증: fold 안에서 전처리하고 고정 후보의 점수·편차·오류를 함께 기록한다.

오늘 남길 것: 앙상블 실험 보고서 + CV 결과 + 중요도 근거 + 재현 가능한 코드와 Git 기록.

다음 수업 · 선택을 마무리하고 최종 평가로

일자	다음 내용	오늘과 연결
10/28 (수)	평가지표·교차검증·Feature Engineering K-Means·최종 Pipeline·과목 실습평가	모델·특성·임계값 정책을 고정한 후 보관한 test로 최종 확인
10/29 (목)	GPU 확인·PyTorch·Tensor·ANN 순전파·손실·역전파	학습과 추론·손실과 평가의 차이를 재사용
10/30 (금)	학습률·Batch·Epoch·과적합 제어 Dropout·BatchNorm·Early Stopping	train/valid 변화와 학습 안정성을 진단

더 복잡한 모델을 쓰기 전에, 데이터·실험 조건·선택 근거를 다른 사람이 재현할 수 있어야 한다.

공식 참고: scikit-learn · Cross-validation

한로 강민준

공식 참고자료 · 트리·앙상블·해석

- sklearn · Decision Trees
- sklearn · RandomForestRegressor
- sklearn · DecisionTreeClassifier
- sklearn · Forest feature importances
- sklearn · DecisionTreeRegressor
- sklearn · Permutation importance
- sklearn · RandomForestClassifier
- sklearn · permutation_importance

공식 문서 링크 · 각 항목 클릭 / 전체 주소는 발표자 노트

한로 강민준

공식 참고자료 · XGBoost·교차검증

- XGBoost · sklearn estimator
- sklearn · Cross-validation
- XGBoost · Parameters
- sklearn · cross_validate
- XGBoost 3.1 · Python API
- sklearn · GridSearchCV
- XGBoost · Installation
- sklearn · Model persistence

공식 문서 링크 · 각 항목 클릭 / 전체 주소는 발표자 노트